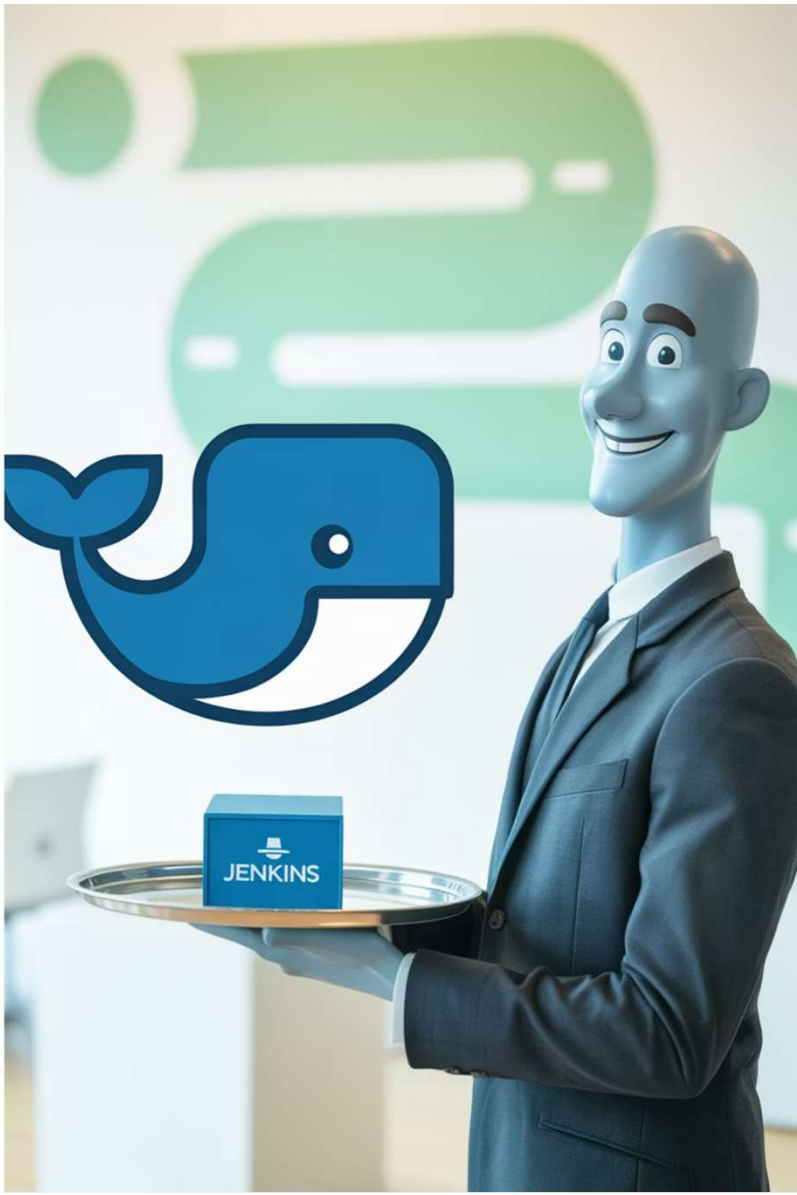
Docker Integration with Jenkins

A comprehensive guide to streamlining your CI/CD pipeline with containerization



Agenda

01

Why Docker with Jenkins?

Understanding the benefits of containerization in CI/CD

02

Installation & Configuration

Setting up Docker on Jenkins server

03

Pipeline Integration

Configuring Docker in Jenkins pipelines

04

Deployment Strategies

Jenkins running in Docker vs building Docker images

05

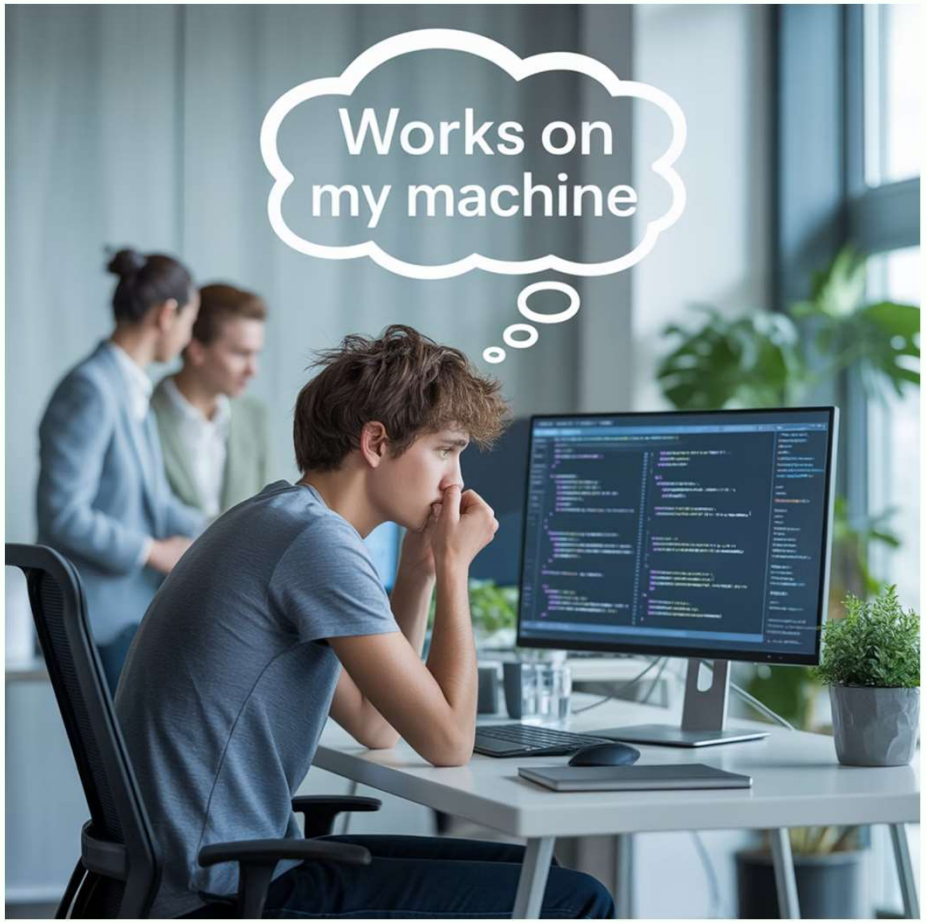
Security Considerations

Best practices for secure Docker integration

Why Use Docker with Jenkins?

Docker containers provide consistent environments across development, testing, and production, eliminating the "it works on my machine" problem. When integrated with Jenkins, they offer:

- Environment consistency across all stages of CI/CD
- Isolated build environments for each job
- Simplified dependency management
- Improved resource utilization
- Faster build and deployment processes



Installing Docker on Jenkins Server

Update Package Repository

```
sudo apt-get updatesudo apt-get install apt-transport-https ca-certificates curl software-properties-common
```

Add Docker's Official GPG Key

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

Install Docker Engine

```
sudo apt-get updatesudo apt-get install docker-ce docker-ce-cli containerd.io
```

Add Jenkins User to Docker Group

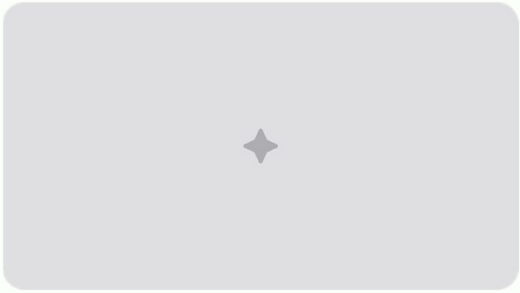
```
sudo usermod -aG docker jenkinssudo systemctl restart jenkins
```

These steps ensure Docker is properly installed and the Jenkins user has appropriate permissions to interact with Docker.

Verifying Docker Installation

After installation, verify Docker is working correctly with Jenkins:

1. Log in to Jenkins server as the Jenkins user
2. Run `docker --version` to confirm Docker is installed
3. Test Docker functionality with `docker run hello-world`
4. Check that Jenkins can execute Docker commands without sudo



⚠ If the Jenkins user cannot run Docker commands, you may need to restart the Jenkins service or the entire server for group changes to take effect.

Configuring Docker Inside Jenkins Pipeline

Jenkins pipelines can leverage Docker in several ways:

Docker Pipeline Plugin

Install the "Docker Pipeline" plugin from Jenkins Plugin Manager to access Docker commands directly in your Jenkinsfile.

Docker Agent

Run pipeline steps inside a Docker container using the `agent { docker { image 'node:14' } }` syntax.

Docker Commands

Execute Docker CLI commands directly in shell steps: `sh 'docker build -t myapp:${BUILD_NUMBER} .'`

These approaches can be combined to create powerful, flexible CI/CD pipelines that leverage containerization throughout the process.

Jenkins Deployment Strategies

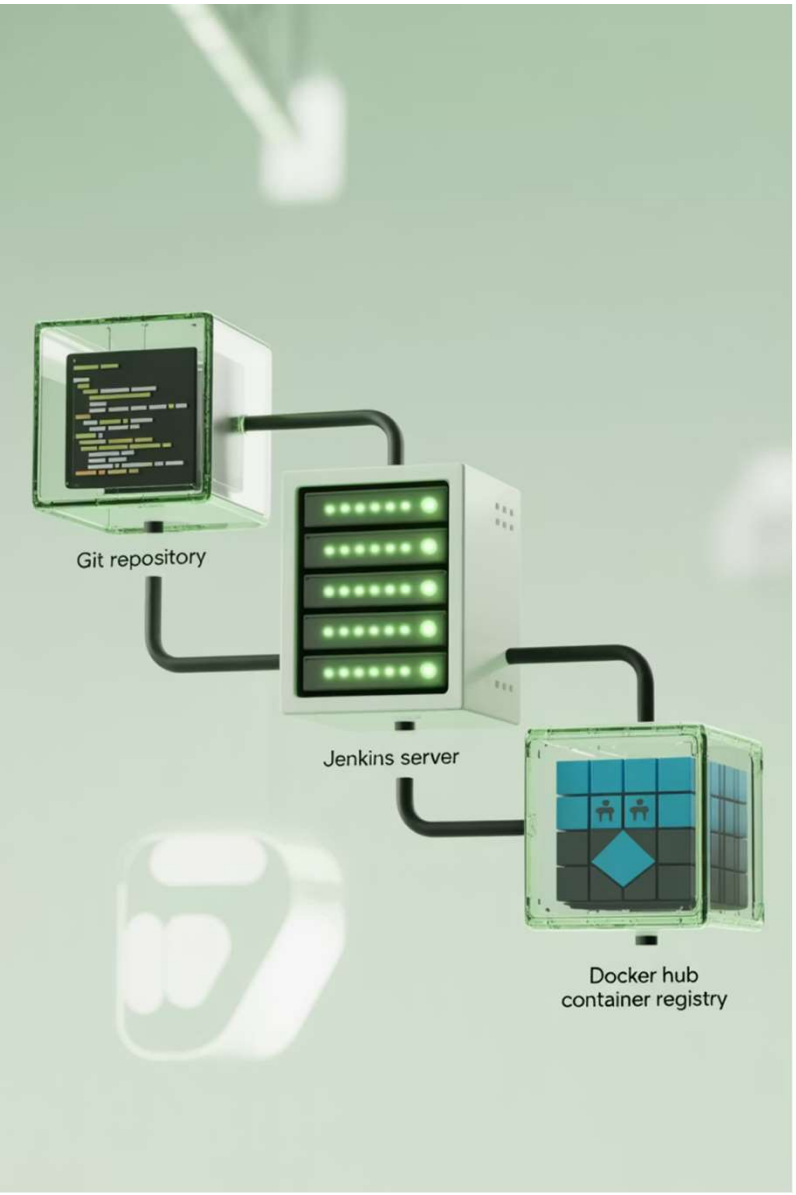
Jenkins running inside Docker

- Jenkins itself runs as a container
- Easy to upgrade and maintain
- Consistent Jenkins environment
- Simple backup and restore
- Requires Docker socket mounting for building images

Jenkins building Docker images

- Traditional Jenkins installation
- Jenkins controls Docker daemon
- Builds and manages containers
- More direct access to host resources
- Requires Docker installation on Jenkins server

Both approaches have their merits - the choice depends on your specific infrastructure needs and security requirements.



Pipeline Example: Overview

A typical Jenkins pipeline for Docker integration follows these key stages:

1. **Clone Repository:** Pull the latest code from your Git repository
2. **Build Docker Image:** Create a container image based on your Dockerfile
3. **Push to Registry:** Upload the built image to DockerHub or another container registry

This workflow enables consistent, repeatable deployments across environments.

Pipeline Example: Jenkinsfile

```
pipeline {    agent any        stages {        stage('Clone') {            steps {                git 'https://github.com/username/repo.git'            }
}        stage('Build') {            steps {                sh 'docker build -t username/myapp:${BUILD_NUMBER} .'            }        }        stage('Push')
{            steps {                withCredentials([string(credentialsId: 'docker-hub-password', variable: 'DOCKERHUB_PASSWORD')]) {                    sh
'docker login -u username -p ${DOCKERHUB_PASSWORD}'                    sh 'docker push username/myapp:${BUILD_NUMBER}'                }            }        }
}}
```

This Jenkinsfile defines a complete pipeline that clones code, builds a Docker image, and pushes it to DockerHub.

Security Considerations

Docker Socket Access

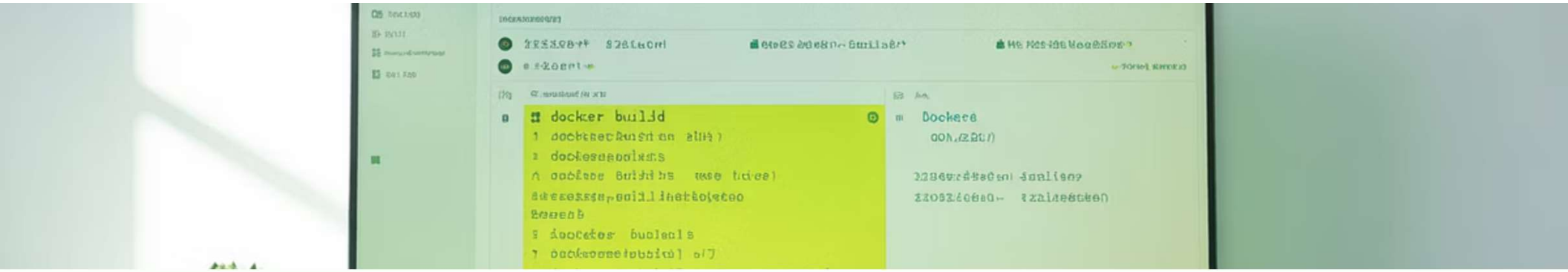
Mounting the Docker socket (`/var/run/docker.sock`) gives containers root-equivalent access to the host. Use with extreme caution and consider alternatives like Docker-in-Docker.

Credential Management

Never hardcode credentials in Dockerfiles or Jenkinsfiles. Use Jenkins Credentials Plugin and pass secrets at runtime.

Image Scanning

Implement vulnerability scanning for container images as part of your pipeline using tools like Trivy, Clair, or Anchore.



Real-Time Example

This screenshot shows a successful Jenkins pipeline that:

- Clones a Git repository containing application code and Dockerfile
- Builds a Docker image with the appropriate tags
- Runs tests inside the container to verify functionality
- Pushes the verified image to DockerHub
- Deploys the application using the new container image

The entire process is automated, consistent, and repeatable across environments.

Advanced Configuration: Docker Compose

For multi-container applications, integrate Docker Compose with Jenkins:

```
stage('Deploy') {  steps {    sh 'docker-compose -f docker-compose.yml up
-d'  }}
```

This allows Jenkins to orchestrate complex application stacks with multiple interconnected containers, managing their lifecycle as a single unit.

Docker Compose enables defining and running multi-container applications, perfect for

Troubleshooting Common Issues

Permission Denied

Symptom: "Got permission denied while trying to connect to the Docker daemon socket"

Solution: Add Jenkins user to docker group and restart Jenkins service

Registry Authentication

Symptom: "unauthorized: authentication required" when pushing images

Solution: Verify Docker Hub credentials in Jenkins and ensure proper login

Resource Constraints

Symptom: Builds fail with "no space left on device"

Solution: Implement regular cleanup of unused images and containers

Proper logging and monitoring are essential for quickly identifying and resolving these common integration issues.

Key Takeaways

1 ***Docker + Jenkins = Powerful CI/CD***

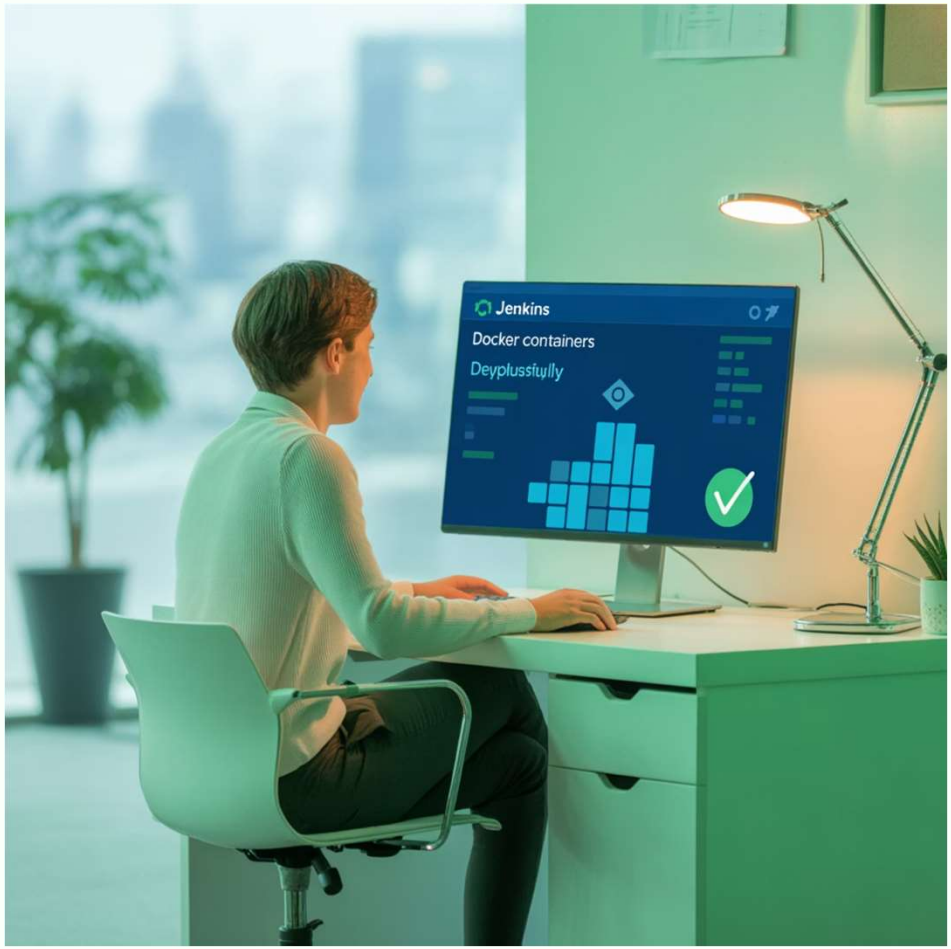
The combination provides consistency, isolation, and scalability for your development pipeline.

2 ***Proper Setup is Critical***

Ensure correct installation and permissions for seamless integration.

3 ***Security Requires Attention***

Follow best practices to avoid exposing your systems to unnecessary risks.



With proper Docker integration, Jenkins becomes an even more powerful tool for modern